UTEC
Universidad Tecnológica

Patrón Singleton

¿Qué es el Patrón Singleton?

El patrón Singleton es un patrón de diseño creacional que se utiliza para restringir la creación de instancias de una clase a un solo objeto. Este patrón asegura que una clase tenga únicamente una instancia y proporciona un punto de acceso global a dicha instancia.

Implementación del Patrón Singleton

La implementación básica del patrón Singleton en Java:

```
public class Singleton {
    // Instancia única de la clase, declarada como estática.
    private static Singleton instanciaUnica;

    // Constructor privado para evitar la instanciación externa.
    private Singleton() {
        // Lógica de inicialización
    }

    // Método público estático para obtener la única instancia de la
clase.
    public static Singleton obtenerInstancia() {
        if (instanciaUnica == null) {
            instanciaUnica = new Singleton();
        }
        return instanciaUnica;
    }

    // Método de ejemplo dentro del Singleton
    public void mostrarMensaje() {
        System.out.println("¡Hola! Esta es la única instancia del
Singleton.");
    }
}
```

UTEC
Universidad Tecnológica

Explicación:

- **Instancia única:** La clase `Singleton` tiene una variable estática llamada `instanciaUnica` que mantiene la única instancia de la clase.
- **Constructor privado:** El constructor de la clase es privado para evitar que otras clases puedan crear instancias de `Singleton` usando el operador `new`.
- **Método de acceso:** El método `obtenerInstancia()` es el punto de acceso global que verifica si la instancia ya ha sido creada. Si no es así, crea una nueva instancia y la devuelve. Si ya existe, simplemente devuelve la instancia existente.

Uso:

```
public class Main {
    public static void main(String[] args) {
        // Obtener la única instancia de Singleton
        Singleton singleton = Singleton.obtenerInstancia();

        // Usar un método de Singleton
        singleton.mostrarMensaje();
    }
}
```

En este ejemplo, la salida será:

```
¡Hola! Esta es la única instancia del Singleton.
```

Usos del Patrón Singleton

El patrón Singleton se utiliza en situaciones donde es crucial que solo exista una única instancia de una clase. Algunos casos comunes incluyen:

- **Conexión a la base de datos:** Mantener una única conexión para gestionar recursos de manera eficiente.
- **Manejadores de configuración:** Cargar configuraciones una vez y compartirlas a lo largo de la aplicación.
- **Gestión de logs:** Mantener un único objeto para registrar eventos o errores, evitando la creación de múltiples archivos o conexiones a sistemas de logging.

UTEC
Universidad Tecnológica

Variaciones del Patrón Singleton y consideraciones de concurrencia

Existen varias formas de implementar el patrón Singleton, cada una con sus ventajas y desventajas, especialmente en entornos concurrentes.

Inicialización Temprana (Eager Initialization)

En esta versión, la instancia del Singleton se crea cuando la clase es cargada por primera vez:

```
public class Singleton {
    // Instancia única creada en el momento de la carga de la clase
    private static final Singleton instanciaUnica = new Singleton();

    // Constructor privado
    private Singleton() {
        // Lógica de inicialización
    }

    // Método de acceso
    public static Singleton obtenerInstancia() {
        return instanciaUnica;
    }
}
```

- **Ventajas:**
    - Simple y fácil de implementar.
    - No requiere sincronización.
- **Desventajas:**
    - No permite el control sobre el momento exacto de la creación de la instancia.
    - Si la instancia nunca se usa, se desperdician recursos.

UTEC
Universidad Tecnológica

Inicialización Perezosa (Lazy Initialization)

La instancia del Singleton se crea la primera vez que se solicita:

```
public class Singleton {
    private static Singleton instanciaUnica;

    private Singleton() {
        // Lógica de inicialización
    }

    public static Singleton obtenerInstancia() {
        if (instanciaUnica == null) {
            instanciaUnica = new Singleton();
        }
        return instanciaUnica;
    }
}
```

- **Ventajas:**
  - La instancia se crea solo si es necesario.
  - Ahorra recursos si nunca se utiliza la instancia.
- **Desventajas:**
  - No es seguro en entornos multi-hilo, lo que puede llevar a la creación de múltiples instancias si dos hilos acceden al método `obtenerInstancia()` al mismo tiempo.

Singleton Seguro para Hilos (Thread-safe Singleton)

Para asegurar la creación de una única instancia en entornos concurrentes, se puede utilizar la sincronización:

```
public class Singleton {
    private static Singleton instanciaUnica;

    private Singleton() {
        // Lógica de inicialización
    }

    public static synchronized Singleton obtenerInstancia() {
```

```
        if (instanciaUnica == null) {
            instanciaUnica = new Singleton();
        }
        return instanciaUnica;
    }
}
```

- **Ventajas:**
  - Es seguro en entornos multi-hilo.
- **Desventajas:**
  - El uso de la palabra clave `synchronized` puede impactar negativamente el rendimiento debido al costo de sincronización.

Doble Verificación de Bloqueo (Double-Checked Locking):

Para optimizar el rendimiento, se puede combinar la sincronización con una verificación doble:

```
public class Singleton {
    private static volatile Singleton instanciaUnica;

    private Singleton() {
        // Lógica de inicialización
    }

    public static Singleton obtenerInstancia() {
        if (instanciaUnica == null) {
            synchronized (Singleton.class) {
                if (instanciaUnica == null) {
                    instanciaUnica = new Singleton();
                }
            }
        }
        return instanciaUnica;
    }
}
```

UTEC
Universidad Tecnológica

- **Ventajas:**
  - Es seguro para hilos y más eficiente que la sincronización simple, ya que solo sincroniza en la primera vez que se crea la instancia.
- **Desventajas:**
  - Es más complejo de implementar.
  - Requiere el uso de la palabra clave `volatile` para asegurar la visibilidad de la instancia única en todos los hilos.

El patrón Singleton es fundamental para situaciones donde es crucial tener una única instancia de una clase, como en el manejo de recursos compartidos o la gestión de configuraciones globales. Su implementación debe considerar el entorno en el que se utilizará, especialmente en aplicaciones concurrentes, para evitar problemas como la creación de múltiples instancias en un entorno multi-hilo.

Ejemplos de Patrón Singleton

1. Conexión a la Base de Datos

En una aplicación, es habitual necesitar una conexión única y compartida a la base de datos para evitar múltiples conexiones innecesarias que puedan saturar los recursos.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBDSingleton {
    private static ConexionBDSingleton instanciaUnica;
    private Connection conexion;
    private String url =
"jdbc:postgresql://localhost:5432/mi_base_de_datos";
    private String usuario = "usuario";
    private String contraseña = "contraseña";

    // Constructor privado para evitar la creación de múltiples
instancias.
    private ConexionBDSingleton() throws SQLException {
```

UTEC
Universidad Tecnológica

```
        try {
            // Cargar el driver de PostgreSQL
            Class.forName("org.postgresql.Driver");
            // Crear la conexión
            this.conexion = DriverManager.getConnection(url, usuario,
contraseña);
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            throw new SQLException("Error al conectar a la base de
datos", e);
        }
    }

    // Método para obtener la única instancia de la conexión
    public static ConexionBDSingleton obtenerInstancia() throws
SQLException {
        if (instanciaUnica == null) {
            instanciaUnica = new ConexionBDSingleton();
        }
        return instanciaUnica;
    }

    // Método para obtener la conexión
    public Connection getConexion() {
        return conexion;
    }
}
```

Uso:

```
public class Main {
    public static void main(String[] args) {
        try {
            ConexionBDSingleton conexion =
ConexionBDSingleton.obtenerInstancia();
            Connection conn = conexion.getConexion();
            // Usar la conexión para operar en la base de datos
        } catch (SQLException e) {
            e.printStackTrace();
```

UTEC
Universidad Tecnológica

```
        }
    }
}
```

2. Manejadores de Configuración

A menudo, las aplicaciones necesitan cargar configuraciones desde un archivo o base de datos y compartir estas configuraciones a lo largo de la aplicación. Usar el patrón Singleton asegura que las configuraciones se carguen una sola vez y estén disponibles globalmente.

```
import java.util.Properties;
import java.io.InputStream;
import java.io.IOException;

public class ConfiguracionSingleton {
    private static ConfiguracionSingleton instanciaUnica;
    private Properties propiedades;

    // Constructor privado para cargar las configuraciones
    private ConfiguracionSingleton() {
        propiedades = new Properties();
        try (InputStream entrada =
getClass().getClassLoader().getResourceAsStream("config.properties")) {
            if (entrada != null) {
                propiedades.load(entrada);
            } else {
                throw new IOException("Archivo de configuración no
encontrado");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    // Método para obtener la única instancia del manejador de
configuraciones
    public static ConfiguracionSingleton obtenerInstancia() {
        if (instanciaUnica == null) {
            instanciaUnica = new ConfiguracionSingleton();
        }
```

```
        return instanciaUnica;
    }

    // Método para obtener una propiedad específica
    public String getPropiedad(String clave) {
        return propiedades.getProperty(clave);
    }
}
```

Uso:

```
public class Main {
    public static void main(String[] args) {
        ConfiguracionSingleton configuracion =
ConfiguracionSingleton.obtenerInstancia();
        String valor =
configuracion.getPropiedad("mi.clave.configuracion");
        System.out.println("Valor de la configuración: " + valor);
    }
}
```

3. Gestión de Logs

Para registrar eventos o errores en una aplicación, es útil tener un único objeto que maneje todos los registros. Esto evita la creación de múltiples archivos o conexiones a sistemas de logging.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class LoggerSingleton {
    private static LoggerSingleton instanciaUnica;
    private PrintWriter escritor;

    // Constructor privado para inicializar el escritor de logs
    private LoggerSingleton() {
```

UTEC
Universidad Tecnológica

```
        try {
            FileWriter archivo = new FileWriter("aplicacion.log", true);
            escritor = new PrintWriter(archivo);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    // Método para obtener la única instancia del logger
    public static LoggerSingleton obtenerInstancia() {
        if (instanciaUnica == null) {
            instanciaUnica = new LoggerSingleton();
        }
        return instanciaUnica;
    }

    // Método para registrar un mensaje en el log
    public void registrar(String mensaje) {
        DateTimeFormatter formato =
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        escritor.println(LocalDateTime.now().format(formato) + " - " +
mensaje);
        escritor.flush();
    }
}
```

Uso:

```
public class Main {
    public static void main(String[] args) {
        LoggerSingleton logger = LoggerSingleton.obtenerInstancia();
        logger.registrar("Aplicación iniciada.");
        // Más código...
        logger.registrar("Aplicación finalizada.");
    }
}
```